

46-debugging-devtools

46. Debugging & DevTools Mastery

Level: □ Intermediate

Jam: 6 (4 sesi × 90 menit)

Prasyarat: Modul 1 (JS Fundamentals), Modul 4 (Web Basics)

Output: Debugging toolkit, debuggable project setup, error monitoring integration

Tujuan Pembelajaran

Setelah modul ini, kamu bisa: - Debug layout, network, dan performance pakai Browser DevTools - Setup dan pakai VS Code Debugger buat breakpoint, watch, call stack - Debug REST API dengan Postman dan analisis waterfall timing - Setup structured logging, baca stack trace, handle unhandled rejection

Materi

Sesi	Topik	File
1	Browser DevTools — Elements, Console, Sources, Network, Performance, Lighthouse	01-browser-devtools.md
2	VS Code Debugger — launch config, breakpoint types, React DevTools, multi-target	02-vscode-debugger.md

Sesi	Topik	File
3	API Debugging — Postman, Thunder Client, waterfall timing, CORS, mock server	03-api-debugging.md
4	Error Tracing & Logging — stack trace, structured logging, unhandled rejection, source maps	04-error-tracing-logging.md

Output Akhir Modul

Debug Dashboard — project Express/Next.js dengan: -
Structured logging (Pino) + log correlation ID - Error handler
middleware + global unhandled rejection - Source maps di
production - Lighthouse audit score ≥ 90

AI Prompt Exercises

Sepanjang modul, latihan pake AI: - “Explain this stack trace — what caused the error?” - “Why is this API call slow? Analyze the waterfall timing” - “Generate a structured logger setup for this Express app” - “Find the memory leak in this code using Chrome DevTools”

1. Browser DevTools

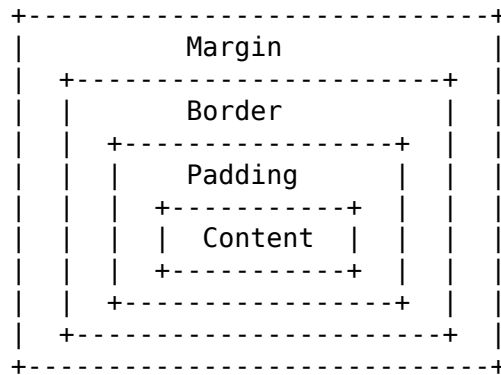
Durasi: 90 menit

1.1 Elements Panel

Inspect dan edit HTML/CSS secara live.

```
// Contoh HTML yg bakal di-debug
const brokenLayout = `
  <div style="display: flex; gap: 8px;">
    <div style="flex: 1; padding: 16px;">Kiri</div>
    <div style="flex: 2; padding: 16px;">Tengah</div>
    <div style="padding: 16px;">Kanan</div>
  </div>
`;
document.body.innerHTML = brokenLayout;
```

Box Model Diagram:



Multipurpose debugging tool.

3

Console Tips: - `$0` — refer ke element terakhir yg di-inspect - `$()` — shortcut `document.querySelector()` - `$$()` — shortcut `document.querySelectorAll()` - `$_` — hasil evaluasi terakhir - `copy(obj)` — copy object ke clipboard - `monitor(functionName)` — log tiap panggilan function

Listener | Sources → Event Listener Breakpoints | Event specific (click, keydown) |

Step Controls: | Button | Shortcut | Fungsi | |---|---|---| | ► Resume | F8 | Lanjut sampai breakpoint berikutnya | | ↘ Step over | F10 | Eksekusi baris, lompat ke baris berikutnya | | ➡ Step into | F11 | Masuk ke function call | | ⬆ Step out | Shift+F11 | Keluar dari function | | ◻ Restart frame | — | Ulangi dari awal function |

Call Stack & Scope: - **Call Stack:** lihat urutan function calls yg menuju ke breakpoint - **Scope:** lihat local, closure, global variables saat itu - **Watch:** tambah expression untuk di-track value-nya

1.4 Network Panel

Analisis traffic HTTP/HTTPS.

```
// Simulasi fetch yang lambat
async function loadDashboard(): Promise<void> {
  const [users, orders, analytics] = await Promise.all([
    fetch("/api/users?page=1"), // Cek timing
    fetch("/api/orders", { signal: AbortSignal.timeout(5000) }), // Timeout
    fetch("/api/analytics/dashboard"), // Mungkin slow query
  ]);

  // Check status
  if (!users.ok) throw new Error(`HTTP ${users.status}`);
}
```

Network Tab Features:

Feature	Fungsi
Waterfall	Timeline tiap request — liat yg blocking
Status column	200 OK, 304 Not Modified, 404, 500
Size column	Transfer size vs resource size
Time column	Total duration request
Filters	XHR, JS, CSS, Img, Media, Font, Doc, WS
Blocking	Right-click → Block request URL — simulasi failed load
Throttling	Simulasi Slow 3G, Fast 3G, Offline
Replay	Right-click → Replay XHR
Copy as Fetch	Copy sebagai fetch() call

Timing Breakdown:

Queued	0.05ms
Stalled	2.34ms

DNS Lookup	1.12ms	
Initial connection	3.45ms	
SSL	2.89ms	
Request sent	0.12ms	
Waiting (TTFB)	345.67ms	← kritis
Content Download	12.34ms	

TTFB (Time To First Byte) > 600ms biasanya perlu optimize backend.

1.5 Performance Panel

Rekam dan analisis runtime performance.

```
// Contoh penyebab performa jelek
function renderLargeList(items: string[]): void {
  const container = document.getElementById("list")!;

  // □ Layout thrashing – forced reflow tiap iterasi
  items.forEach((item) => {
    const el = document.createElement("div");
    el.textContent = item;
    container.appendChild(el);
    console.log(container.offsetHeight); // forced reflow!
  });
}

// □ Batch operation
function renderLargeListFixed(items: string[]): void {
  const container = document.getElementById("list")!;
  const fragment = document.createDocumentFragment();

  items.forEach((item) => {
    const el = document.createElement("div");
    el.textContent = item;
    fragment.appendChild(el);
  });

  container.appendChild(fragment); // single reflow
}
```

Performance Panel Key Metrics: | Metric | Arti | |—|—|—| | FPS
 | Frames Per Second — target 60fps | | CPU | CPU usage — kalo 100%
 terus ada masalah | | Layout Shift | Visual stability — Cumulative Lay-
 out Shift (CLS) | | Long Tasks | Task > 50ms blocking main thread | |
 Flame Chart | Visualisasi call stack per frame |

Rekaman: 1. Buka Performance panel 2. Klik ● Record 3. Lakukan

action yg mau diukur 4. Klik ■ Stop 5. Analisis flame chart

1.6 Lighthouse

Audit otomatis untuk kualitas web app.

```
// Lighthouse via CLI
// npm install -g lighthouse
// lighthouse https://example.com --view
```

```
// Lighthouse via Node.js
```

```
import lighthouse from "lighthouse";
import * as chromeLauncher from "chrome-launcher";
```

```
async function runAudit(url: string): Promise<void> {
  const chrome = await chromeLauncher.launch({ chromeFlags: ["--headless"] });
  const options = {
    logLevel: "info",
    output: "json",
    onlyCategories: ["performance", "accessibility", "seo", "best-practices"],
    port: chrome.port,
  };

  const result = await lighthouse(url, options);
  console.log("Performance:", result.lhr.categories.performance.score);
  console.log("Accessibility:", result.lhr.categories.accessibility.score);
  console.log("SEO:", result.lhr.categories.seo.score);
  console.log("Best Practices:", result.lhr.categories["best-practices"].score);

  await chrome.kill();
}
```

Lighthouse Categories: | Category | Score Target | Berat | |-----|
|-----| | Performance | ≥ 90 | Largest Contentful Paint, First Input Delay, CLS | | Accessibility | ≥ 90 | ARIA labels, contrast ratio, keyboard nav | | Best Practices | ≥ 90 | HTTPS, no console errors, no vulnerabilities | | SEO | ≥ 90 | Meta tags, viewport, crawlability | | PWA | ≥ 80 | Service worker, manifest, offline |

Latihan

1. **Debug Layout Broken** — Buat HTML dengan flex layout rusak (element overflow, wrong alignment). Pakai Elements panel buat diagnose dan fix tanpa reload.
2. **Find Slow API** — Buka web app (atau JSONPlaceholder) di Network panel. Identifikasi request paling lambat. Analisis waterfall

— apa yg bikin lama? DNS? TTFB? Download?

3. **Audit with Lighthouse** — Run Lighthouse audit di blog/page favorit. Catat score tiap kategori. Tulis 3 rekomendasi improvement dari hasil audit.
4. **Console Mastery** — Buka console di halaman manapun. Coba: `$0`, `$()`, `$$()`, `copy()`, `console.table()`, `console.time()`. Dokumentasi hasilnya.
5. **Performance Record** — Rekam performance halaman. Identifikasi 1 long task. Jelaskan apa yg terjadi dan gimana cara fixnya.

2. VS Code Debugger

Durasi: 90 menit

2.1 Launch Configuration

Setup debugger di VS Code.

.vscode/launch.json:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug Node.js",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/dist/index.js",
      "preLaunchTask": "tsc: build - tsconfig.json",
      "outFiles": ["${workspaceFolder}/dist/**/*.js"],
      "sourceMaps": true,
      "skipFiles": ["<node_internals>/**"]
    },
    {
      "name": "Attach to Process",
      "type": "node",
      "request": "attach",
      "processId": "${command:PickProcess}",
      "restart": true,
      "localRoot": "${workspaceFolder}",
      "remoteRoot": "/app"
    }
  ]
}
```

```

    "name": "Debug Jest Tests",
    "type": "node",
    "request": "launch",
    "runtimeExecutable": "${workspaceFolder}/node_modules/.bin/jest",
    "args": ["--runInBand", "--testTimeout", "30000"],
    "console": "integratedTerminal",
    "internalConsoleOptions": "neverOpen"
  },
  {
    "name": "Debug with nodemon",
    "type": "node",
    "request": "launch",
    "runtimeExecutable": "nodemon",
    "args": ["--exec", "node", "--inspect-brk", "dist/index.js"],
    "restart": true,
    "console": "integratedTerminal"
  }
]
}

```

Launch vs Attach:

Mode	Kapan Pakai
launch	VS Code yg jalankan program (start debugging)
attach	Program udah jalan — tinggal sambung debugger

Key Config Fields:

Field	Fungsi
program	Entry point file
preLaunchTask	Task yg dijalankan sebelum debug (misal: compile)
outFiles	Lokasi output JS untuk source maps
sourceMaps	Enable source maps — biar bisa debug .ts asli
skipFiles	File yg di-skip pas step (biar ga masuk node_modules)
restart	Restart otomatis pas file berubah (kombinasi nodemon)
envFile	File .env untuk environment variables

2.2 Breakpoint Types

VS Code support 6 jenis breakpoint.

// Contoh code untuk breakpoint demo

// 1. LINE BREAKPOINT – klik kiri di gutter

```

function calculateTotal(items: { price: number; qty: number }[]): number {
  let sum = 0;
  for (const item of items) { // ← line breakpoint di sini
    sum += item.price * item.qty;
  }
  return sum;
}

// 2. LOGPOINT – log tanpa pause
function processPayment(amount: number): string {
  // Logpoint: "Processing payment: ${amount}"
  // Hasil console tanpa stop eksekusi
  const formatted = `Rp ${amount.toLocaleString("id-ID")}`;
  return `Paid: ${formatted}`;
}

// 3. CONDITIONAL BREAKPOINT
function findUser(id: number): User | null {
  // Right-click → Add Conditional Breakpoint
  // Condition: id < 0
  // Hanya pause kalo id negatif (invalid)
  return users.find((u) => u.id === id) ?? null;
}

// 4. HIT COUNT BREAKPOINT
function processBatch(items: string[]): void {
  for (let i = 0; i < items.length; i++) {
    // Hit count: 3
    // Pause pas iterasi ke-3
    console.log(`Processing ${items[i]}`);
  }
}

// 5. FUNCTION BREAKPOINT – set di Breakpoints panel
function sendEmail(to: string, subject: string): void {
  // Function breakpoint: sendEmail
  // Pause tiap kali sendEmail dipanggil
  console.log(`Sending email to ${to}: ${subject}`);
}

// 6. INLINE BREAKPOINT – break di tengah expression
const result = someAsyncFunction(); // ← inline breakpoint di dalam expression

```

Breakpoint Summary Table:

Type	Cara Set	Use Case
Line Logpoint	Klik gutter Right-click → Logpoint	Pause di baris spesifik Debug tanpa pause ("printf debugging")
Conditional	Right-click → Conditional	Pause kalo condition true
Hit Count Function	Right-click → Hit Count Breakpoints panel → Tambah	Pause pas iterasi ke-n Set breakpoint di function manapun
Inline	Shift+F9	Break di tengah expression

2.3 Debug Console

Evaluasi expression dan execute command saat debug.

// Saat paused di breakpoint, bisa eval di Debug Console:

```
// 1. Cek nilai variable
// > user.name
// "Budi"
```

```
// 2. Execute function
// > calculateTotal(items)
// 250000
```

```
// 3. Modifikasi value
// > user.role = "admin"
// "admin"
```

```
// 4. Async expression
// > await fetchUser(1)
// { id: 1, name: "Budi" }
```

```
// 5. Import dan panggil
// > require("lodash").cloneDeep(items)
// [...]
```

```
// 6. Error handling
// > try { JSON.parse(invalid) } catch(e) { e.message }
// "Unexpected token"
```

Shortcuts:

Shortcut	Fungsi
F5	Start / Continue debug
Shift+F5	Stop debug
F10	Step over
F11	Step into
Shift+F11	Step out
Ctrl+Shift+F9	Delete all breakpoints
Ctrl+F5	Start without debugging

2.4 Watch, Variables, Call Stack

Panel penting di Debug view.

```
interface Order {
  id: string;
  userId: string;
  items: OrderItem[];
  total: number;
  status: "pending" | "confirmed" | "shipped";
}

async function processOrder(orderId: string): Promise<void> {
  const order = await fetchOrder(orderId); // Pause di sini
  // Di Debug panel:
  //   VARIABLES: order = { id: "ORD-001", userId: "USR-1", ... }
  //   WATCH:
  //     - order.items.length
  //     - order.status === "pending"
  //     - typeof order.total
  //   CALL STACK:
  //     - processOrder (this file)
  //     - handleRequest (server.ts:45)
  //     - router.handle (router.ts:120)
  //     - <anonymous> (server.ts:10)

  const user = await fetchUser(order.userId);
  // Sekarang CALL STACK bertambah 1 frame lagi

  const success = await executePayment(order, user);
  if (!success) {
    throw new Error("Payment failed");
  }
}
```

Watch Expressions: - Bisa pake expression kompleks: `items.reduce((s,i)`

=> s + i.price, 0) - Method chaining: users.filter(u => u.active).map(u => u.name) - Type check: typeof x === "string" ? x.length : "N/A"

Variables Panel: - **Local:** variable dalam scope function saat ini - **Closure:** variable dari outer function yg di-capture - **Global:** variable global (window/globalThis) - **Module:** variable module scope

2.5 Debug React

React DevTools dan debugging specific.

React DevTools Extension:

```
// Install:
// Chrome: React Developer Tools
// Firefox: React DevTools

// Component tree – inspect props dan state
function UserCard({ user }: { user: User }) {
  const [expanded, setExpanded] = useState(false);
  // Di React DevTools:
  //   <UserCard>
  //     props: { user: { id: 1, name: "Budi" } }
  //     hooks:
  //       State(0): expanded = false
  //       State(1): setExpanded = f ()
  return (
    <div onClick={() => setExpanded(!expanded)}>
      <h3>{user.name}</h3>
      {expanded && <p>{user.email}</p>}
    </div>
  );
}
```

React DevTools Features:

Feature	Fungsi
Components tree	Lihat hierarchy komponen
Props & State	Inspect nilai saat ini
Highlight updates	Highlight komponen yg re-render
Profiler	Record render performance — flame chart komponen
Source	Klik komponen → buka source code di Sources tab

React Profiler:

```

// Buka React DevTools → Profiler tab
// Klik Record → lakukan action → Stop

// Analisis flame chart:
// - Warna: hijau (cepat) → kuning → merah (lambat)
// - Durasi render tiap komponen
// - Ranked chart – komponen paling lambat di atas

// Fix re-render:
const UserCard = React.memo(function UserCard({ user }: { user: User }) {
  // Hanya re-render kalo props user berubah
  return <div>{user.name}</div>;
});

```

2.6 Multi-Target Debugging

Debug multiple proses sekaligus.

Compound Launch Config:

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Server",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/server/src/index.ts",
      "preLaunchTask": "tsc: build - server/tsconfig.json",
      "outFiles": ["${workspaceFolder}/server/dist/**/*.js"]
    },
    {
      "name": "Client",
      "type": "chrome",
      "request": "launch",
      "url": "http://localhost:5173",
      "webRoot": "${workspaceFolder}/client/src",
      "sourceMaps": true
    },
    {
      "name": "Worker",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/worker/src/index.ts",
      "preLaunchTask": "tsc: build - worker/tsconfig.json",
      "outFiles": ["${workspaceFolder}/worker/dist/**/*.js"]
    }
  ]
}

```

```

    }
  ],
  "compounds": [
    {
      "name": "Full Stack: Server + Client",
      "configurations": ["Server", "Client"],
      "preLaunchTask": "npm: dev"
    },
    {
      "name": "All Services: Server + Client + Worker",
      "configurations": ["Server", "Client", "Worker"],
      "stopAll": true
    }
  ]
}

```

Multi-target Debugging Tips: - stopAll: true — stop semua proses pas satu berhenti - Tiap proses punya Debug Console sendiri - Switch antar proses di Call Stack dropdown

Latihan

1. **Debug Infinite Loop** — Tulis kode dengan infinite loop. Set breakpoint, identifikasi exit condition yg salah. Fix and verify.
2. **Debug Async Waterfall** — Buat 3 function async berantai (A → B → C). Set breakpoint di tiap function. Step through pake Step Into dan Step Over. Catat urutan eksekusi.
3. **Debug Race Condition** — Tulis kode dengan Promise.all dan race condition di shared variable. Set conditional breakpoint untuk track perubahan variable.
4. **Setup Compound Config** — Buat launch.json dengan Server (Node.js) + Client (Chrome debug). Jalankan compound debug.
5. **React DevTools Practice** — Buka React project (atau buat minimal), record Profiler, temukan komponen yg re-render tidak perlu.

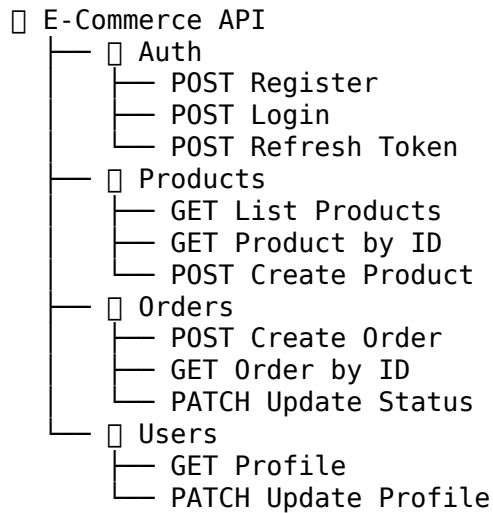
3. API Debugging

Durasi: 90 menit

3.1 Postman — Collections & Environment

Setup Postman untuk API debugging.

Collection Structure:



Environment Variables:

```
{
  "local": {
    "base_url": "http://localhost:3000",
    "api_key": "dev-key-123"
  },
  "staging": {
    "base_url": "https://staging-api.example.com",
    "api_key": "staging-key-456"
  },
  "production": {
    "base_url": "https://api.example.com",
    "api_key": "prod-key-789"
  }
}
```

Variable Usage di Request:

URL: {{base_url}}/api/products
Header: Authorization: Bearer {{token}}
Body JSON: {"userId": "{{user_id}}"}
}

Pre-request Script (JavaScript):

```
// Auto-set token sebelum request
const loginRequest = {
```

```

url: `${pm.environment.get("base_url")}/auth/login`,
method: "POST",
header: { "Content-Type": "application/json" },
body: {
  mode: "raw",
  raw: JSON.stringify({
    email: pm.environment.get("test_email"),
    password: pm.environment.get("test_password"),
  }),
},
};

pm.sendRequest(loginRequest, (err, res) => {
  if (err) {
    console.error("Login failed:", err);
    return;
  }
  const token = res.json().token;
  pm.environment.set("token", token);
  console.log("Token set:", token.substring(0, 20) + "...");
});

```

Test Script:

```

// Assertions otomatis setelah response diterima
pm.test("Status code is 200", () => {
  pm.response.to.have.status(200);
});

pm.test("Response time < 500ms", () => {
  pm.expect(pm.response.responseTime).to.be.below(500);
});

pm.test("Has valid token", () => {
  const json = pm.response.json();
  pm.expect(json).to.have.property("token");
  pm.expect(json.token).to.match(/^eyJ/); // JWT starts with eyJ
});

pm.test("Content-Type is JSON", () => {
  pm.response.to.have.header("Content-Type", /json/);
});

// Data-driven assertion
const schema = {
  type: "object",
  properties: {

```

```

    id: { type: "number" },
    name: { type: "string", minLength: 1 },
    email: { type: "string", pattern: "^\\S+@\\S+$" },
    role: { type: "string", enum: ["admin", "user"] },
  },
  required: ["id", "name", "email", "role"],
};

pm.test("Response matches schema", () => {
  const json = pm.response.json();
  pm.expect(json).to.be.an("object");
  pm.expect(json).to.have.all.keys("id", "name", "email", "role");
  pm.expect(json.email).to.match(/^\\S+@\\S+$/);
});

```

Collection Runner:

```

# Jalankan semua request di collection
Collections → E-Commerce API → Run
- Iterations: 3 (loop)
- Delay: 1000ms (antar request)
- Data: test-data.csv (data-driven)
- Save Responses: ✓
- Keep variable values: ✓

```

3.2 Thunder Client / REST Client (VS Code)

Alternatif Postman langsung di VS Code.

Thunder Client: - Extension: Thunder Client - Config via thunder-client.json di .vscode/

REST Client (humao.rest-client): - File-based: buat .rest atau .http file

```

### Variables
@baseUrl = http://localhost:3000/api
@token = {{$dotenv TOKEN}}

### Login
POST {{$baseUrl}}/auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "secret123"
}

```

```

#### Get Profile (auto-use token from login)
GET {{baseUrl}}/users/profile
Authorization: Bearer {{token}}
Accept: application/json

```

```

#### Create Product
POST {{baseUrl}}/products
Authorization: Bearer {{token}}
Content-Type: application/json

```

```

{
  "name": "Kopi Gayo",
  "price": 45000,
  "category": "Minuman",
  "stock": 100
}

```

```

#### Check Response
# @name createProduct
# HTTP 201
# {
#   "id": 42,
#   "name": "Kopi Gayo",
#   "price": 45000
# }

```

REST Client Features: | Feature | Cara | |———|———| | Variables |
 @varName = value | | Environment | {{\$.dotenv VAR}} dari .env | |
 Response testing | #### block dgn expected status | | Request history |
 .http file — version-controlled | | Auth | Bearer, Basic, Digest, OAuth2
 |

3.3 Network Tab Analysis

Deep dive waterfall timing.

```

// Server-side simulation of slow endpoint
import express from "express";
const app = express();
const PORT = 3000;

// Endpoint sengaja lambat – buat latihan debug timing
app.get("/api/products", async (req, res) => {
  // Simulasi slow DB query
  await new Promise((r) => setTimeout(r, 800));

```

```
// Simulasi slow serialization
const products = Array.from({ length: 1000 }, (_, i) => ({
  id: i + 1,
  name: `Product ${i + 1}`,
  price: Math.random() * 100000,
  description: "x".repeat(5000), // large payload
}));

res.json({
  data: products,
  total: products.length,
  page: 1,
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Waterfall Timing Breakdown:

Request: GET /api/products

Queueing	0.12ms	□
Stalled	2.34ms	□
DNS Lookup	1.12ms	□
Initial Connection	3.45ms	□
SSL	2.89ms	□
Request Sent	0.12ms	□
Waiting (TTFB)	812.45ms	□ ← PROBLEM
Content Download	234.56ms	□

Total 1056.93ms

What Each Phase Means:

Phase	Max	Arti
DNS Lookup	< 50ms	Resolve domain name → IP
TCP Connect	< 100ms	Three-way handshake
TLS Handshake	< 100ms	SSL/TLS negotiation
TTFB	< 600ms	Server processing time
Content Download	depends	Ukuran response / bandwidth
Total	< 3000ms	End-to-end duration

Debugging Slow Phases: - **DNS slow (> 100ms):** Ganti DNS provider (Cloudflare 1.1.1.1, Google 8.8.8.8) - **TTFB slow (> 600ms**

): Optimize server — database query, caching, CDN - **Content Download slow**: Compress response (gzip/brotli), pagination, smaller payload - **Connection slow**: Use HTTP/2, keep-alive, CDN edge

3.4 Error Debugging

Parse dan debug berbagai HTTP error.

```
// API error response format
interface ApiErrorResponse {
  status: number;
  code: string;
  message: string;
  details?: Record<string, string[]>;
  timestamp: string;
  requestId: string;
}

// 400 Bad Request – validation error
async function createUser(data: unknown): Promise<void> {
  const res = await fetch("/api/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });

  if (!res.ok) {
    const error: ApiErrorResponse = await res.json();

    switch (res.status) {
      case 400:
        // Parse validation errors
        console.error("Validation failed:", error.details);
        // { "email": ["Email already exists"], "name": ["Name is required"] }
        break;

      case 401:
        // Unauthorized – redirect to login
        console.error("Auth failed:", error.message);
        window.location.href = "/login";
        break;

      case 403:
        // Forbidden – insufficient permissions
        console.error("Access denied:", error.message);
        break;
    }
  }
}
```

```

    case 404:
      // Not Found
      console.error("Resource not found:", error.message);
      break;

    case 429:
      // Rate limited
      const retryAfter = res.headers.get("Retry-After") ?? "60";
      console.warn(`Rate limited. Retry after ${retryAfter}s`);
      break;

    case 500:
      // Server error
      console.error("Server error:", error);
      // Log requestId for debugging
      console.error("Request ID:", error.requestId);
      break;

    default:
      console.error(`Unexpected error ${res.status}:`, error);
  }
  throw new Error(error.message);
}
}

```

Debugging CORS:

// CORS error: "No 'Access-Control-Allow-Origin' header is present"
// Solution: Server harus set header

```

import express from "express";
import cors from "cors";

const app = express();

// Opsi 1: Allow all (development only)
app.use(cors());

// Opsi 2: Whitelist specific origins
app.use(
  cors({
    origin: ["https://myapp.com", "http://localhost:5173"],
    methods: ["GET", "POST", "PUT", "DELETE", "PATCH"],
    allowedHeaders: ["Content-Type", "Authorization"],
    credentials: true,
    maxAge: 86400, // cache preflight 24 jam
  })
);

```

```

    })
  );

  // Opsi 3: Dynamic origin
  app.use(
    cors({
      origin: (origin, callback) => {
        const allowed = ["https://myapp.com", "http://localhost:5173"];
        if (!origin || allowed.includes(origin)) {
          callback(null, true);
        } else {
          callback(new Error("Not allowed by CORS"));
        }
      },
    })
  );

```

CORS Debug Checklist:

1. ☐ Apakah preflight OPTIONS request ada di Network tab?
2. ☐ Apakah response punya Access-Control-Allow-Origin header?
3. ☐ Apakah origin ada di allowed list?
4. ☐ Apakah method ada di Access-Control-Allow-Methods?
5. ☐ Apakah custom headers ada di Access-Control-Allow-Headers?
6. ☐ Apakah credentials: true diset kalo pake cookies?

3.5 Mock Server

Simulasi API response tanpa backend real.

Postman Mock Server:

```

{
  "mock": {
    "collections": ["E-Commerce API"],
    "environment": "staging",
    "mock_url": "https://e6f3a2b4-e8c7-4a5d-9b1c-3d2e1f4a5b6c.mock.pstmn.io",
    "delay": 200,
    "examples": [
      {
        "request": "GET /api/products",
        "response": {
          "status": 200,
          "body": {
            "data": [
              {
                "id": 1,
                "name": "Mock Product A",
                "price": 50000,

```

```

        "stock": 100
      }
    ],
    "total": 1
  }
},
{
  "request": "GET /api/products/:id",
  "response": {
    "status": 404,
    "body": {
      "error": "Product not found"
    }
  }
}
]
}
}

```

Postman Interceptor: - Capture request dari browser langsung - Forward ke Postman collection - Bisa mock response - Berguna buat debug API dari SPA

Latihan

1. **Debug API Pagination** — Panggil endpoint paginated (/api/products?page=1&limit=10). Cek apakah total page sesuai, apakah ada duplicate/cursor issue. Debug pake Postman collection runner.
2. **Debug Auth Error** — Setup 3 environment (local, staging, prod). Pakai pre-request script buat auto-login dan set token. Debug kasus dimana token expired dan harus refresh.
3. **Debug Timeout** — Buat endpoint sengaja slow (setTimeout 10s). Panggil dari Postman dengan timeout 5s. Debug error timeout, analisis waterfall.
4. **CORS Debug** — Buat frontend sederhana (HTML+JS) fetch dari origin berbeda. Catat CORS error. Fix dengan setting cors middleware di server.
5. **Mock Server Setup** — Buat Postman mock server untuk 3 endpoint. Test dari REST Client .http file.

4. Error Tracing & Logging

Durasi: 90 menit

4.1 Stack Trace Anatomy

Baca dan pahami stack trace.

```
// Contoh error dengan stack trace lengkap
function divide(a: number, b: number): number {
  if (b === 0) throw new Error("Division by zero");
  return a / b;
}

function calculateAverage(numbers: number[]): number {
  const sum = numbers.reduce((acc, n) => acc + n, 0);
  return divide(sum, numbers.length);
}

function processGrades(grades: number[]): void {
  // [] Array kosong → length = 0 → divide by zero
  const avg = calculateAverage(grades);
  console.log("Average:", avg);
}

// Panggil
processGrades([]);
```

Stack Trace Output:

```
Error: Division by zero
    at divide (stack.ts:3:11)
    at calculateAverage (stack.ts:9:12)
    at processGrades (stack.ts:15:15)
    at Object.<anonymous> (stack.ts:19:1)
    at Module._compile (node:internal/modules/cjs/loader:1256:14)
    at Module._extensions (node:internal/modules/cjs/loader:1310:10)
    at Module.load (node:internal/modules/cjs/loader:1115:32)
    at Module._load (node:internal/modules/cjs/loader:958:12)
    at Function.executeUserEntryPoint (node:internal/modules/run_main:81:12)
    at Object.<anonymous> (node:internal/modules/run_main:81:12)
```

Stack Trace Anatomy:

```
Error: Division by zero          ← Error name + message
    at divide (stack.ts:3:11)     ← File:line:column
      ↑       ↑ ↑
      function name   line column
```

```

    at calculateAverage (stack.ts:9:12)
    at processGrades (stack.ts:15:15)
    at Object.<anonymous> (stack.ts:19:1) ← Entry point
    at Module._compile ...
    at Module._extensions ...
    at Module.load ...
                                ↓
                                Node.js internal
                                (ignore for debugging)

```

Async Stack Trace:

```

async function fetchUser(id: string): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  if (!res.ok) throw new Error(`HTTP ${res.status}: User not found`);
  return res.json();
}

async function loadDashboard(userId: string): Promise<void> {
  try {
    const user = await fetchUser(userId);
    renderProfile(user);
  } catch (err) {
    // Async stack trace menunjukkan dari mana async call berasal
    console.error("Dashboard failed:", err);
  }
}

loadDashboard("invalid-id");

```

Async Stack Trace Output:

```

Error: HTTP 404: User not found
    at fetchUser (async.ts:4:11)
    at async loadDashboard (async.ts:10:22) ← async boundary
    at async Object.<anonymous> (async.ts:17:1)

```

Node.js flag untuk async stack trace lebih detail:

```

# Async stack trace – biar ga ilang async context
node --async-stack-traces dist/index.js

```

4.2 Error Types & Handling

Berbagai jenis error dan cara handle.

```

// 1. TRY-CATCH – synchronous error
function parseJSON(input: string): unknown {
  try {
    return JSON.parse(input);
  } catch (err) {
    if (err instanceof SyntaxError) {

```

```

        console.error("Invalid JSON:", (err as Error).message);
        return null;
    }
    throw err; // rethrow kalo bukan yg dihandle
}
}

// 2. ERROR-FIRST CALLBACK (Node.js legacy)
import fs from "fs";
fs.readFile("/path/to/file.txt", "utf-8", (err, data) => {
    if (err) {
        console.error("Failed to read file:", err.message);
        return;
    }
    console.log("File content:", data);
});

// 3. PROMISE REJECTION
import { readFile } from "fs/promises";

async function readConfig(): Promise<Config> {
    try {
        const data = await readFile("config.json", "utf-8");
        return JSON.parse(data);
    } catch (err) {
        // Handle both file-not-found AND parse error
        if (err instanceof SyntaxError) {
            throw new Error("Config file is malformed");
        }
        if ((err as NodeJS.ErrnoException).code === "ENOENT") {
            throw new Error("Config file not found");
        }
        throw err;
    }
}

// 4. PROMISE CHAIN .catch()
fetch("/api/users")
    .then((res) => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
    })
    .then((data) => processUsers(data))
    .catch((err) => {
        console.error("Request failed:", err);
        // Continue with fallback
    })

```

```

        return [];
    });

    // 5. EVENT EMITTER ERROR
    import { EventEmitter } from "events";

    class DatabaseConnection extends EventEmitter {
        connect(): void {
            // Simulate connection error
            process.nextTick(() => {
                this.emit("error", new Error("Connection refused"));
            });
        }
    }

    const db = new DatabaseConnection();
    db.on("error", (err: Error) => {
        console.error("Database error:", err.message);
        // Reconnect logic here
    });
    db.connect();

    // IMPORTANT: EventEmitter ERROR tanpa listener → crash process!

```

Error Handling Decision Table:

Scenario	Strategy
User input validation	Try-catch + return user-friendly message
Network request	Promise .catch() or async/await try-catch
File I/O	Handle specific error codes (ENOENT, EACCES)
Third-party library	Wrap in try-catch, log, rethrow as known error
EventEmitter	Wajib ada error listener — crash kalo ga ada
Unhandled rejection	process.on("unhandledRejection") — see 4.4

4.3 Structured Logging

Log yang bisa dibaca mesin — JSON format, levels, correlation ID.

Setup dengan Pino:

```

import pino from "pino";
import { randomUUID } from "crypto";
import http from "http";

// Logger instance
const logger = pino({

```

```

level: process.env.LOG_LEVEL ?? "info",
transport:
  process.env.NODE_ENV === "development"
    ? {
        target: "pino-pretty",
        options: { colorize: true, translateTime: "SYS:standard" },
      }
    : undefined,
redact: ["req.headers.authorization", "req.body.password", "user.token"],
serializers: {
  req: pino.stdSerializers.req,
  res: pino.stdSerializers.res,
  err: pino.stdSerializers.err,
},
});

// Correlation ID – track request across services
function generateCorrelationId(): string {
  return randomUUID();
}

// Middleware Express + Pino
import express from "express";
const app = express();

app.use((req, res, next) => {
  const correlationId =
    (req.headers["x-correlation-id"] as string) ?? generateCorrelationId();
  res.setHeader("x-correlation-id", correlationId);

  // Child logger dengan correlation ID
  req.log = logger.child({ correlationId, service: "api-gateway" });
  next();
});

// Usage in route handler
app.get("/api/users/:id", async (req, res) => {
  const log = (req as any).log;

  log.info({ userId: req.params.id }, "Fetching user");

  try {
    const user = await fetchUser(req.params.id);
    log.info({ userId: req.params.id, status: "found" }, "User fetched");
    res.json(user);
  } catch (err) {

```

```

    log.error(
      { err, userId: req.params.id },
      "Failed to fetch user"
    );
    res.status(500).json({ error: "Internal server error" });
  }
});

```

Log Levels:

Level	Nilai	Kapan Pakai
fatal	60	App will crash / can't continue
error	50	Error yg butuh immediate attention
warn	40	Something unexpected, app still works
info	30	Normal operation — request processed
debug	20	Development detail — query, timing
trace	10	Very verbose — full payload dump

JSON Log Output:

// Contoh log entry

```

{
  "level": 30,
  "time": 1720123456789,
  "pid": 12345,
  "hostname": "server-01",
  "correlationId": "alb2c3d4-e5f6-7890-abcd-ef1234567890",
  "service": "api-gateway",
  "userId": "usr_42",
  "msg": "User fetched",
  "status": "found"
}

```

Winston Alternative:

```

import winston from "winston";

const winstonLogger = winston.createLogger({
  level: "info",
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: "user-service" },
  transports: [

```

```

    new winston.transports.File({
      filename: "logs/error.log",
      level: "error",
      maxsize: 5242880, // 5MB
      maxFiles: 5,
    }),
    new winston.transports.File({
      filename: "logs/combined.log",
      maxsize: 5242880,
      maxFiles: 10,
    }),
  ],
});

if (process.env.NODE_ENV !== "production") {
  winstonLogger.add(
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.simple()
      ),
    })
  );
}

```

Pino vs Winston:

Aspek	Pino	Winston
Speed	□ 2-3x faster	Slower (more features)
Bundle size	~10KB	~50KB
Transports	Stream-based	Multi-transport built-in
JSON default	Yes	Optional
Ecosystem	Growing	Mature, more plugins

4.4 Unhandled Rejection & Global Error Handler

Tangkap error yg tidak tertangani.

```

// GLOBAL UNHANDLED REJECTION
process.on("unhandledRejection", (reason: unknown, promise: Promise<unknown>) => {
  console.error("UNHANDLED REJECTION:", reason);

  // Log ke file / monitoring
  logger.fatal(
    {

```

```

        err: reason instanceof Error ? reason : new Error(String(reason)),
        promise,
    },
    "Unhandled Promise Rejection"
);

// Opsi A: Crash – exit process biar restart manager (PM2/Docker)
process.exitCode = 1;

// Opsi B: Recover – kalo error non-fatal
// process.exitCode tetap 0, tapi log warning
});

// UNCAUGHT EXCEPTION
process.on("uncaughtException", (err: Error, origin: string) => {
    console.error("UNCAUGHT EXCEPTION:", err);

    logger.fatal(
        {
            err,
            origin,
            pid: process.pid,
        },
        "Uncaught Exception – process will exit"
    );

    // Uncaught exception → state mungkin corrupted
    // Harus exit, biar process manager restart
    process.exit(1);
});

// WARNING – deprecation, multiple listeners
process.on("warning", (warning: Error) => {
    logger.warn(
        { warning: warning.message, stack: warning.stack },
        "Process warning"
    );
});

```

Best Practices:

Practice	Kenapa
Jangan recover dari uncaughtException Use process manager (PM2, Docker restart)	State app mungkin corrupted — lebih aman restart Biar auto-restart pas crash

Practice	Kenapa
Log correlation ID di error	Traceability antar service
Source maps di production	Stack trace readable walau minified
Error boundary di UI	Jangan sampe white screen
Alert monitoring	Sentry, Datadog, New Relic integration

Source Maps untuk Production:

```
// tsconfig.json
{
  "compilerOptions": {
    "sourceMap": true,
    "inlineSources": true
  }
}

// Atau di Node.js runtime
// node --enable-source-maps dist/index.js

// Di package.json
{
  "scripts": {
    "start": "node --enable-source-maps dist/index.js",
    "start:prod": "node --enable-source-maps dist/index.js"
  }
}

// Upload ke error monitoring
// npx @sentry/cli sourcemaps upload --org=myorg --project=myproject dist/
```

Sentry Integration:

```
import * as Sentry from "@sentry/node";

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV ?? "development",
  release: `v${process.env.npm_package_version}`,
  tracesSampleRate: 1.0,
  integrations: [
    new Sentry.Integrations.Http({ tracing: true }),
    new Sentry.Integrations.Express({ app }),
  ],
});
```

```
});

// Express error handler
app.use(Sentry.Handlers.errorHandler());

// Manual capture
try {
  dangerousOperation();
} catch (err) {
  Sentry.captureException(err);
}
```

Latihan

1. **Setup Structured Logging** — Buat Express app dengan Pino. Implement correlation ID di middleware. Log setiap request (method, url, duration, status code). Output JSON file.
2. **Trace Error Across Async Chain** — Buat 3 function async (A → B → C) dimana C throw error. Pastikan stack trace show full chain. Test dengan dan tanpa `--async-stack-traces`.
3. **Read Minified Stack Trace** — Build TypeScript dengan `--minify`. Dapatkan stack trace minified. Trace balik ke source asli pake source maps.
4. **Global Error Handler** — Tambah `unhandledRejection` dan `uncaughtException` handler di app. Buat 2 kasus: crash (exit) dan recover (warning). Log dengan struktur JSON.
5. **Sentry Setup** — Integrasi Sentry di Express app. Capture manual error, verify di Sentry dashboard.